Securing Credential Sequence Verification

Mamunur Akand, Reihaneh Safavi-Naini, and Marc Kneppers

Abstract. Credentials are used to verify a user's identity and attributes and form the basis of securing user access to the system resources. Users obtain credentials and store them on their (mobile) devices, and present them when needed. Anonymous credentials protect the user's identity, and ensure unlinkability of multiple showing of the credential. In this paper, we consider a setting where a user is issued multiple credentials in sequence (e.g., for completing courses), and credential subsequences must be presented in order of issuance. We focus on the anonymous credential system where information such as the time of issuing is hidden for anonymity, or settings where there is no global clock and issuing time information is not recorded. We propose a novel order-preserving Proof-of-Credential-Subsequence ($\mathbb{P}o\mathbb{C}\$$) system called KROM that allows a user that is potentially untrusted, to present a subsequence of their locally stored credentials to a verifier, while the relative chronological order of issuance is preserved. We formalize the security and privacy of KROM and present two constructions: a basic one that is based on Merkle trees and one with batched verification that significantly improves the efficiency of the system. We use KROM to construct an anonymous order-preserving proof-of-location-subsequence system and prove its security. The system enables users to selectively present a subsequence of their visited locations to a verifier or an auditor. The main challenge that is addressed is to ensure that the location information that must be in plaintext, does not breach privacy when used in sequence.

Keywords: Credential sequence · Order-preserving credential showing· Proof-of-location

1 Introduction

Credential systems are vital for securing information communication systems. A digital credential, in its basic form, is a digitally signed document issued by a trusted credential issuing authority. It can be verified by all verifiers who have access to the public key of the issuing authority. Users need to prove their claimed attribute or property to the credential issuer, and the issued digital credential enables them to prove the corresponding claim to other verifiers. Anonymous credential systems [14, 21, 32, 13] allow users to prove their properties or attributes without revealing their identities. This is increasingly important, mandated by privacy laws, and aligns with users' desire to minimize their digital footprint on the Internet.

Credentials like driver's licenses or passports are typically limited to one active copy at a time. They can be re-issued, but only after the previous one

expires and becomes invalid. On the other hand, credentials such as course credits for learners or access tokens for system resources can be collected, stored on users' devices, and presented when needed, potentially alongside other credentials and according to specific policies. One commonly used policy is to require credentials to be presented in a particular order, such as the order of issuance. However, in anonymous credential systems, the information within the credential is not directly accessible to verifiers. It exists as claims that can only be verified during credential presentation. As a result, demonstrating the relationship between two credentials, such as one being issued before the other, poses a challenge.

In this paper, we address the challenge of verifying the chronological order of credentials, specifically focusing on a particular type called *proof-of-location* (*pol*) credentials. These credentials attest to a user's presence at a specific location and contain location information in plaintext. While our approach is general and applicable to various scenarios where credential sequences require preservation of chronological order without synchronized clocks, we specifically examine the case of *pol* credentials. *pol* credentials are tokens generated by a Proof-of-Location ($\mathbb{P}o\mathbb{L}$) system that utilizes location infrastructure employing technologies like WiFi and Bluetooth. This system determines a user's location or proximity to landmarks and issues *pol* tokens certifying their presence. These tokens can be presented individually or in sequence to demonstrate traversal of a path [22, 24, 26]. The location infrastructure is a distributed system with issuing nodes potentially located in geographically distant places and different time zones. In privacy-preserving *pol*s, the exact time of *pol* issuance is hidden, making it impossible to directly use time information for verifying the order of issuance.

Credential storage: Credentials must be readily available when needed while maintaining their privacy and ensuring that only the owner can access and present them. One approach to achieving accessibility and preserving the order of credential issuance is to store the credentials in a trusted distributed database and retrieve them as required. Secure time stamping by the database can be utilized to preserve and prove the order. A public blockchain serves as an elegant implementation of such a database, leveraging the sequential nature of its public immutable data structure and global accessibility to securely demonstrate the order of credential issuance. However, this solution comes with a high cost, such as 81 USD per 1 kilobyte of data storage on Ethereum[1]. Moreover, the public nature of the database may inadvertently leak additional information, even if individual credential tokens are anonymous[2]. A more cost-effective solution is for users to store their issued credentials on their own devices and present them as desired. This approach aligns with the concept of *self-sovereign* identity, which has gained significant interest and adoption in academia and industry [34, 15, 28, 17, 20]. It empowers users to manage their own credentials. However, when (anonymous) credentials must be presented in a specific order,

[1] 1 kilobyte requires 640k gas or 0.032 ETH [29], equivalent to 81 USD based on the current Ethereum rate of 2,545 USD on January 12, 2024.

[2] Note that anonymous credentials can still contain plaintext information, such as location

challenges arise due to the user's ability to reorder credentials on their device. While tampering with a single credential may be infeasible due to the security of the underlying credential system, the user can modify the presentation sequence by rearranging the credentials. If explicit order information cannot be obtained from the credential itself, ensuring the integrity of the credential presentation and its compliance with the required order becomes uncertain.

Our work. In this paper, we propose KROM[3], a system that enables users to locally store their credentials while ensuring order-preserving credential sequence presentation based on the chronological order of issuance. Credentials are stored by the user on their device, and a concise updatable fingerprint capturing the issuing sequence is stored on a public blockchain. This fingerprint serves as an accumulator for the credential sequence, allowing verification of any subsequence against the stored fingerprint. Additionally, we design and demonstrate the security of a privacy-preserving proof-of-location-subsequence ($\mathbb{P}\circledcirc\mathbb{L}\$$) system that adapts KROM for enabling the order-preserving presentation of a subsequence of locations visited by the user. Our contributions are as follows.

i. Proof-of-Credential-Subsequence ($\mathbb{P}\circledcirc\mathbb{C}\$$): Model and Construction. We propose a *blockchain-based snapshot system* called KROM, that employs a trusted *snapshot manager* with access to a public blockchain, that receives a copy of the credential when it is issued to a user (simultaneously) and publishes/updates a short fingerprint called *snapshot* on the blockchain. The snapshot can be used by the verifiers to verify the correctness of any presented credential subsequence. We define $\mathbb{P}\circledcirc\mathbb{C}\$$ and formalize its security using a game-based approach. We define two security properties for $\mathbb{P}\circledcirc\mathbb{C}\$$: *subsequence membership* and *selective reveal*. A subsequence of a sequence $\mathbf{x} = (x_1, x_2 \cdots x_n)$ is a sequence $\mathbf{x}' = (x_{i_1}, x_{i_2}, \cdots x_{i_j})$, where $i_u < i_v$ when $u < v$ and x_{i_ℓ} is an element of $\mathbf{x}$. *Subsequence membership* ensures that only subsequences of the full credential sequence can be verified successfully, and any presentation of a credential sequence that does not correspond to a well-formed subsequence will be rejected. The *selective reveal* property ensures that the subsequence verification protocol does not leak any information about credentials that are not part of the claimed subsequence.

Construction. We present a cryptographic construction of $\mathbb{P}\circledcirc\mathbb{C}\$$ using a Merkle hash tree to generate a concise fingerprint for a credential sequence. The user and a trusted *Snapshot Manager (SM)* each receive a copy of an issued credential and employ the same algorithm to insert the credential's hash value into their local Merkle tree. The trees grow identically, ensuring that credentials are stored in the order of issuance and generating a secure index for each credential. The SM publishes a *snapshot*, a short string containing information required to verify node inclusion in the tree, on the blockchain. The SM also stores necessary information about the tree's state for accurate tree updates. The algorithms for tree updates and snapshot generation can be found in Section 3.1. When presenting a credential subsequence to a verifier, the user includes the corresponding

[3] KROM is the Bengali word for "sequence" or "order" among a set of items.

Merkle proofs, demonstrating membership and relative positions of tree leaves, using their current Merkle tree. The verifier can then verify each membership proof and utilize the index information to establish the order. We prove that the proposed scheme satisfies the two required properties of $\mathbb{P}o\mathbb{C}\$$ systems.

Batched verification. The cost of verification in the above basic $\mathbb{P}o\mathbb{C}\$$ scheme grows linearly with the length of the credential subsequence. In Section 3.1 we present $\mathbb{P}o\mathbb{C}\$+$ that groups a number of Merkle proofs into one, to achieve higher efficiency in verification. This is achieved by replacing hash values in the lowest layer of the Merkle tree with values that are generated by using a polynomial commitment scheme. The cost saving in using polynomial commitment has the trade-off of needing to store up to t credentials in a plain sequence form on the blockchain and as part of the snapshot, before combining them into a single committed value.

ii. Privacy-preserving Proof-of-Location-Subsequence ($\mathbb{P}o\mathbb{L}\$$): model and construction. We utilize $\mathbb{P}o\mathbb{C}\$$ to construct a system for showing a sequence of anonymous *pol*s. Each *pol* conceals user identity information while containing plaintext location information. In the $\mathbb{P}o\mathbb{L}$ construction described in the Appendix 4, a *pol* is issued to a user when they are in close proximity to the issuer. A *pol* consists of $(cmt, PK_I, \ell oc_I)$, where cmt represents the user's committed identity information, PK_I is the issuer's public key, and ℓoc_I denotes the issuer's location. The cryptographic protocol of *pol* showing enables the verifier to accept or reject a *pol*. Preserving user privacy is crucial in *pol* systems, ensuring that verifiers do not learn any other information except what is provided in the plaintext in the *pol*. Maintaining privacy during the showing of a sequence of *pol*s is vital, as the sequence of locations reveals the trace of a user's movements. However, defining privacy in this context is challenging due to the possibility of common entries (*pol*s) in two subsequences, which could enable a verifier to link multiple showings of the same user's credentials.

We refer to a secure and private $\mathbb{P}o\mathbb{L}$ system that supports order preserving *pol* sequence verification as Proof-of-Location-Subsequence ($\mathbb{P}o\mathbb{L}\$$) system, and define two properties for it: i) *Integrity* that ensures only correctly issued *pol* sequences with correct issuing order, can pass the verification algorithm, and any tampering with *pol*s or their sequence will be detected with overwhelming probability; and ii) *unlinkability* that ensures that colluding verifiers cannot mix and match parts of different verification sessions of users to infer information about *pol* sequence for a user. In Section 5 we present a proof-of-location subsequence scheme that uses KROM as a building block, and together with an anonymous $\mathbb{P}o\mathbb{L}$ system (presented in Section 4) guarantees integrity and unlinkability. We define the security and privacy of the system by extending the corresponding notions in KROM and including the location information in *pol* to prove our construction's security in this model.

In section 6, we study the feasibility of integrating the scheme into a public blockchain called NameCoin (namecoin.org), which is a fork of Bitcoin and allows the registration of a key-value pair on the blockchain for a small fee. We

analyze the latency and storage restrictions of NameCoin and discuss potential workarounds.

Proofs of theorems are in the Appendix B.

2 Preliminaries

Credential System $\mathbb{CS}$. A credential system has four entities: a *trusted authority* TA that generates public and private parameters for the system, *user* U who collects credentials from issuers and presents them to verifiers, *issuer* I issues credential to the user after verifying the validity of the request, and *verifier* V checks the validity of the user presented credential and may provide some service based on the verification decision. $\mathbb{CS}$ has three algorithms:

- $\texttt{Init}(1^k) \rightarrow (sk_I, pk_I)$ where TA generates a public/private key pair sk_I, pk_I for issuer I.
- $\texttt{Issue}(sk_U, pk_I, sk_I, m) \rightarrow x$ or $\perp$: I issues credential x on a message m to user U.
- $\texttt{Verify}(sk_U, [m], x, pk_I) \rightarrow b$: verifier checks the validity of the credential x presented by user U.

A basic $\mathbb{CS}$ provides *Unforgeability*: only a valid issuer should be able to issue a credential to a valid user, and *Non-transferability*: a user U must not be able to successfully claim the possession of a credential that was issued to a user U' (assuming U' does not share their secret key with U).

Merkle hash Tree $\mathbb{MT}$. Merkle hash tree [27] is a binary tree where each node in the tree has at most two child nodes; if the number of leaves is not a power of 2, the binary tree will not be complete. Each non-leaf node stores the hash value of the data element associated with that leaf. For the i^{th} node, the node value will be $h_i = H(x)$ which is the hash value of the data element x. A parent node is the hash of the concatenation of its two child nodes (e.g. for the nodes with index 1 and 2, with hash values h_1 and h_2, respectively, the parent node is computed as $H(h_1||h_2)$. A *Merkle proof* for a data element x that corresponds to a leaf node, is a set of nodes in the tree that constructs a path in the tree to the root, starting from the sibling node of x, followed by the sibling of the parent node of x, and moving up to the higher levels until it reaches the topmost layer. A Merkle hash tree has three algorithms:

- $\texttt{GenerateRoot}(\mathbf{x}) \rightarrow R$: Given a set of ordered data elements $\mathbf{x} = \{x_1, ..., x_n\}$, outputs the root of the tree R that is constructed as above.
- $\texttt{GenerateProof}(x, R) \rightarrow p$: Given an element x and a tree with root R, outputs a Merkle proof p for the data element, or $\perp$ if x does not correspond to a leaf of the Merkle tree.
- $\texttt{VerifyProof}(x, p, R) \rightarrow \{0, 1\}$: Given a data element x, a Merkle proof p and a Merkle root R, outputs 1 if p is valid, or 0 otherwise.

Accumulator $\mathbb{A}$. Accumulates a set of values into a single published value A, such that it is possible to prove for any member of the set that it is incorporated into this published value. We use an accumulator scheme that was proposed by Benaloh and de Mare [7]. The scheme has the following algorithms:

- $\texttt{SetupAcc}(1^k) \rightarrow par$ generates system parameters.
- $\texttt{Acc}(par, L) \rightarrow A$ generates the accumulator value (A) for a set of prime numbers L.
- $\texttt{GenWit}(par, c, L) \rightarrow w$ generates a witness for a value c.
- $\texttt{VerAcc}(par, A, c, w) \rightarrow \{0, 1\}$ verifies that an input value c with the associated witness w is incorporated in the accumulator value A.

We construct an updater function that replaces an accumulated value, that is $\texttt{UpdateAcc}(par, L, c, c') \rightarrow A'$, replaces $c \in L$ with c', and updates the accumulator value A accordingly. Assuming the strong RSA assumption holds, the scheme satisfies the strong collision-resistance property.

Polynomial Commitment Scheme $\mathbb{PC}$. This scheme allows a committer to commit to a polynomial with a short string such that it can be used by a verifier to confirm the correctness of the claimed evaluations of the polynomial. We use the $\mathbb{PC}$ scheme in [23] that has the following algorithms:

- $\texttt{Setup}(1^k, t) \rightarrow (pk_c, sk_c)$ generates public/private keypair for the scheme.
- $\texttt{Commit}(pk_c, \phi(x)) \rightarrow (cmt, d)$ generates the commitment cmt for the polynomial $\phi(x)$, and (optionally) decommitment information d.
- $\texttt{CreateWitness}(pk_c,\ \phi(x),\ i,\ d) \rightarrow (i, \phi(i), w_i)$ outputs a value w_i that is a witness for $\phi(i)$ being a correct evaluation at index i of $\phi(x)$.
- $\texttt{VerifyEval}(cmt, i, \phi(i), w_i) \rightarrow \{1, 0\}$ verifies if $\phi(i)$ is a correct evaluation at index i of the polynomial that is committed in cmt.
- $\texttt{CreateWitnessBatch}(pk_c, \phi(x), B) \rightarrow (r(x), w_B)$ generates a batch witness value w_B and some polynomial $r(x)$ used in batch verification for a set of indices B, $|B| < t$ in a polynomial $\phi(x)$.
- $\texttt{VerifyEvalBatch}(pk_c, cmt, B, r(x), w_B) \rightarrow \{1, 0\}$ verifies witness w_B for a set of indices B in the polynomial committed in cmt.

This commitment scheme is *statistically hiding* and *computationally binding*, under the SDH assumption [8].

Designated verifier Signature $\mathbb{DVS}$. This scheme works as a traditional digital signature scheme with an additional property that enables the holder of the signature to *designate* the signature to a *designated* verifier using the verifier's public key and allowing them to verify the signature, but not be able to convince a third party about the validity of the signature. We use this scheme in credential showing to achieve *unlinkability*. We use the scheme proposed in [33], that consists of the following algorithms:

- $\texttt{GenCP}(1^k) \rightarrow cp$ is common parameter generation algorithm.

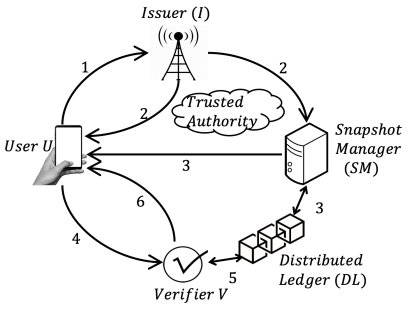
- $\texttt{GenSKey}(cp) \rightarrow (sk_1, pk_1)$ is the signer key generation algorithm.
- $\texttt{GenVKey}(cp) \rightarrow (sk_3, pk_3)$ is the verifier key generation algorithm.
- $\texttt{Sig}(sk_1, m) \rightarrow \sigma$ is the signing algorithm.
- $\texttt{Ver}(pk_1, m, \sigma) \rightarrow \{0, 1\}$ is the public verification algorithm.
- $\texttt{Des}(pk_1, pk_3, m, \sigma) \rightarrow \hat{\sigma}$ is the designation algorithm.
- $\texttt{DVer}(pk_1, sk_3, m, \hat{\sigma}) \rightarrow \{0, 1\}$ is the designated verification algorithm.
- $\texttt{KeyRegV}(cp) \rightarrow (pk_3, \{0, 1\})$ is the verifier key-registration protocol.

In addition to unforgeability against chosen message attack [19], the scheme provides "*DV-unforgeability*" that prevents attacks performed by malicious designators to fool the designated verifier with a forged signature. To prevent a designated verifier from convincing a third party about the signer of the signature, the above scheme also ensures the "*non-transferability privacy*" property.

Other schemes. We also use the digital signature ($\mathbb{DS}$) scheme (in [11]) that provides existential unforgeability, a public key encryption scheme ($\mathbb{E}$) that provides indistinguishability under chosen-ciphertext (IND-CCA) attack [30], a zero-knowledge proof of knowledge ($\mathbb{ZKPK}$) protocol for the knowledge committed value, and a commitment scheme $\mathbb{C}$ with (computational) hiding and binding properties [16]. Details of these schemes are in Appendix A.

3 Proof-of-Credential-Subsequence ($\mathbb{P}\circ\mathbb{CS}$)

We consider credentials that are issued over time and form a time-ordered sequence, and aim to provably and efficiently preserve the order of possible subsequences at the time of credential showing. A $\mathbb{P}\circ\mathbb{CS}$ scheme generates and maintains a short string, called *snapshot*, for a sequence of credentials $\mathbf{x}$ that allows a verifier to verify that a presented sequence of credentials $\mathbf{y}$ is a well-formed subsequence of $\mathbf{x}$. We do not assume a global or synchronized clock.



Trusted Authority provides secret and public keys to the system entities. 1. U makes a credential request to I. 2. I generates a credential x and sends to both U and SM. 3. SM updates (or initiates) snapshot s and sends to U and writes on DL. 4. U generates subsequence proof π for a sequence of credentials $\mathbf{y}$, and sends $\pi, \boldsymbol{y}, s$ to V. 5. V reads DL to obtain the current snapshot s. 6. V verifies the subsequence claim of U and sends $accept/reject$ decision to U.

Fig. 1. $\mathbb{P}\circ\mathbb{CS}$ system model

$\mathbb{P}\circ\mathbb{CS}$ *model and definitions.* A $\mathbb{P}\circ\mathbb{CS}$ scheme is built over a secure credential scheme $\mathbb{CS}$, and in addition to the four entities in the credential system (i.e.,

trusted authority TA, issuer I, user U, and verifier V), the $\mathbb{P}\text{o}\mathbb{C}\$$ scheme, includes a *Snapshot Manager* SM that has access to a public distributed ledger DL system. SM initializes a snapshot $snapsot_U$ for the user U, publishes it on the DL, and updates it as it receives credentials newly issued to U. See Figure 1 for detail.

We assume DL to be a distributed ledger (blockchain) that is publicly accessible and is maintained by a secure consensus algorithm. In $\mathbb{P}\text{o}\mathbb{C}\$$, the snapshot manager SM is the only entity that writes to DL. We define two interfaces for DL: $DL.write(x)$ that writes a value x on DL, and $DL.read()$ that returns the latest value of x written on DL.

Definition 1 (Subsequence). *Let* $\mathbf{x} = \{x_1, \ldots, x_m\}$ *and* $\mathbf{y} = \{y_1, \ldots, y_n\}$ *denote two sequences of credentials.* $\mathbf{y}$ *is a subsequence of* $\mathbf{x}$ *if for every pair of credentials* $(y_i, y_j) \in \mathbf{y}$ *such that* $i < j$*, there exists a pair of credentials* $(x_{i'}, x_{j'}) \in \mathbf{x}$*,* $x_{i'} = y_i$*,* $x_{j'} = y_j$*, such that* $i' < j'$*.*

A $\mathbb{P}\text{o}\mathbb{C}\$$ system proves the validity of a presented sequence of credentials with respect to their issuance order.

Assumptions. Credential issuer I and snapshot manager SM are trusted. User U is dishonest and can claim a false subsequence of credentials, which may include forged credentials and/or credentials with invalid order. Verifier V is honest but curious wanting to infer private information of credentials.

We assume a secure and private communication channel between I and SM, that can be implemented by cryptographic protocols such as TLS (*Transport Layer Security)* protocol.

Definition 2 (Proof-of-Credential-Subsequence $\mathbb{P}\text{o}\mathbb{C}\$$). *A* $\mathbb{P}\text{o}\mathbb{C}\$$ *scheme has the following five algorithms:*

1. $\texttt{Init}(1^k) \rightarrow pk_I, sk_I$*: TA generates* (pk_I, sk_I) *by calling* $\mathbb{CS}$.`Init` *algorithm of the credential scheme* $\mathbb{CS}$ *(see Section 2).*
2. $\texttt{Issue}(sk_U, pk_I, sk_I, m) \rightarrow x$ *or* $\perp$*: Has two stages:*
   (a) $\texttt{Gen}(sk_U, pk_I, sk_I) \rightarrow x$*:* $\mathbb{CS}$.`Issue` *protocol is run between* U *and* I*. The generated credential* x *is sent to both* U *and* SM*.*
   (b) $\texttt{Store}(x, pk_I)$*: Both the user* U *and the* SM *runs* $\mathbb{CS}$.`Verify` *to verify the credential that is received from* I*, store the credential* x *in their respective ordered lists of credentials* L^U_{cred} *and* L^{SM}_{cred}*, according to the order of credential reception from* I*.*
3. $\texttt{Update}(Q, s') \rightarrow s$ *or* $\perp$*: On input* Q*, that is a data structure containing a sequence of credentials, and a snapshot* s'*,* SM *generates the updated snapshot* s*. For the first request, the snapshot is initialized.* SM *writes* s *on* DL*.*
4. $\texttt{ProofGen}(\mathbf{x}, \mathbf{y}) \rightarrow \pi$*:* U *runs this algorithm on the credential sequences* $\mathbf{x} = \{x_1, \ldots, x_n\}$ *and* $\mathbf{y} = \{y_1, \ldots, y_m\}$*,* $m \leq n$*, and outputs a proof* π *that can be used to prove that* $\mathbf{y}$ *is a subsequence of* $\mathbf{x}$*.*
5. $\texttt{Verify}(\pi, \mathbf{y}, s, pk_I) \rightarrow b$*:* U *sends* $(\pi, \mathbf{y})$ *pairs to* V*. At* V*, if* $\mathbb{CS}$.`Verify` *succeeds for all* $y \in \mathbf{y}$ *and* s *is a snapshot that is recorded on* DL*, the verifier* V *will verify* π *for the credential sequence* $\mathbf{y}$*, and using snapshot* s*.* V *outputs* $b = 1$ *(accept) or* $b = 0$ *(reject).*

We use a game-based approach to define the security of $\mathbb{P}\circ\mathbb{C}\$$.

Security game. The game is between a challenger C and an adversary A, with the challenger initializing the system and providing oracle access to different parts of the system. A has access to four types of *oracle queries*:

- $Corrupt(P)$: Adversary A queries to corrupt party P, which can either be user U, or verifier V. C sets the code of P following A's instruction. All protocol transcripts involving P and internal states of P are passed to A. C appends P to a list named $L_{corrupt}$.
- $Issue_{pk_I}(m)$: C simulates the `Issue` protocol on message m, and returns the generated credential x to A. C appends the credential x in an indexed list called L_{issue}. Credentials in this list are indexed in ascending order based on their order of issuance.
- $Update(\mathbf{x}, s')$: A queries to C requesting to update the snapshot s', for the sequence of credentials $\mathbf{x}$. C simulates the snapshot manager SM and runs the $\texttt{Update}(\mathbf{x}, s')$ protocol with U, and returns updated snapshot s, or $\perp$ if update failed. If s is the output, C appends $(\mathbf{x}, s)$ to a list named L_{update}.
- $Verify(\pi, s, \mathbf{y})$: Adversary sends this query to C requesting to verify a proof π claiming that $\mathbf{y}$ is a subsequence of a credential sequence represented by the snapshot s. C simulates the verifier and runs `Verify` of the $\mathbb{P}\circ\mathbb{C}\$$ scheme with user U, and returns either 1 or 0 to the adversary. If 1 is output, C appends $(\pi, s, \mathbf{y})$ in a list named L_{verify}.

Definition 3 ($\mathbb{P}\circ\mathbb{C}\$$ Game). *For a security parameter k, the security game between the challenger C and the adversary A has three stages:*

1. *Setup: C generates public and private key pairs for the credential issuer I and user U by running* $\texttt{Setup}(1^k)$ *of $\mathbb{P}\circ\mathbb{C}\$$ scheme. C also initializes the lists $L_{corrupt}$, L_{issue}, L_{update}, and L_{verify}.*
2. *Queries: Adversary makes a polynomial number of oracle queries listed above.*
3. *Adversary outputs: A sequence of credentials $\mathbf{y} = (y_1, \ldots, y_m)$*

Security Properties. We define two security properties, *subsequence membership* and *selective reveal* for $\mathbb{P}\circ\mathbb{C}\$$ schemes. *Subsequence membership* requires that the adversary should not be able to prove that a credential sequence $\mathbf{y}$ is a subsequence of a credential sequence $\mathbf{x}$, while it is not.

Property 1 (Subsequence Membership). Consider a $\mathbb{P}\circ\mathbb{C}\$$ game where $P = U$ in the *Corrupt* query and A outputs a sequence of credentials $\mathbf{y}$. A wins the game if there exists an entry $(\pi, s, \mathbf{y}) \in L_{verify}$ such that

1. There exists an entry $(\mathbf{x}, s) \in L_{update}$, and,
2. There exists a pair of credentials $(y_i, y_j) \in \mathbf{y}$ such that $y_i \neq y_j, i < j$, and,
3. There does not exist a pair of credentials $(x_{i'}, x_{j'}) \in \mathbf{x}$, $x_{i'} = y_i$, $x_{j'} = y_j$, such that $i' < j'$.

A $\mathbb{P}\circ\mathbb{C}\$$ scheme satisfies *subsequence membership* if the advantage of A to win this game is negligible.

The second property, *selective reveal*, captures the privacy of credentials when presented to colluding verifiers. It states that even if $(N-1)$ out of N credentials in a sequence are opened and their content (referred to as "messages") is revealed to an adversary, the adversary cannot gain any knowledge about the remaining credential. The credential information pertains to the content of the message that is signed by the issuer and may vary depending on the credential system. The message can be provided by the user to the issuer for verification, or it can involve claims such as a location claim verified through an interactive subprotocol (e.g. a distance bounding protocol [6, 9, 2, 3]). To define the *selective reveal* property, we consider a one-bit message randomly chosen by the user with a probability of 1/2. This means that the adversary can only make a random guess regarding the information contained in a credential message.

Property 2 (Selective Reveal). Consider a $\mathbb{P}\circ\mathbb{C}\$$ game with $P = V$ in the *Corrupt* query. We replace steps ii and iii of the game with the following steps:

1. A chooses an integer N, denoting the number of credentials to be issued to the user U, and a value $1 \leq i \leq N$, the index that denotes the position of the only credential that will not be opened to A.
2. C chooses a random bit $m \leftarrow \{0, 1\}$, and simulates $\texttt{Issue}(sk_U, pk_I, sk_I, m)$ of the $\mathbb{P}\circ\mathbb{C}\$$ scheme between user U and the credential issuer I, and follows it up with $\texttt{Update}(\{m\}, s)$. This step is repeated N times.
3. C sends all credentials except the i-th one to A. Let b denote the message (random bit) on which the i-th credential is issued.
4. A is allowed to make $Verify$ queries, for a polynomial number of times.
5. A outputs a bit $\hat{b}$.

If $\hat{b} = b$, A wins the game. A $\mathbb{P}\circ\mathbb{C}\$$ scheme achieves *selective reveal* if the advantage of A to win this game is negligible.

3.1 KROM: A $\mathbb{P}\circ\mathbb{C}\$$ Construction

Consider a snapshot manager SM that uses a buffer of length n that stores the last n credentials in the credential sequence, and uses a FIFO strategy for updating the buffer when a new credential arrives. A queue Q with two functions, $Enqueue(x)$ and $Dequeue()$, implements the buffer. $Enqueue(x)$ appends the latest credential x to the buffer, and $Dequeue()$ removes the oldest credential from the queue.

Snapshot generation using Merkle hash Tree. We utilize a Merkle hash tree $\mathbb{MT}$ (see Section 2) to construct the snapshot s. In our construction, each data element in the Merkle tree is represented as $\langle i, x\rangle$, where x denotes the i-th credential in the inserted sequence. The value of the i^{th} leaf node (N_i) is computed as $h_i = H(i||x)$. This ensures that the leaves of the tree securely store the index of the corresponding node.

The snapshot s employs a compact representation of the Merkle tree. This representation only requires $log(n)$ nodes from the tree, where n represents the

number of leaves in the tree. It provides the necessary information for two purposes: i) computing the root R of the Merkle tree, and ii) accurately updating the snapshot by computing the updated R when a new credential arrives and needs to be accumulated in the tree.

The Merkle tree $\mathbb{MT}$ is defined as $\langle [hashes], treeSize \rangle$, where $[hashes]$ denotes an array of hash values, and $treeSize$ represents the number of leaf nodes in the Merkle tree. Figure 2 illustrates the compact representation of a Merkle hash tree and how it evolves with the arrival of new credentials and the growth of the tree.

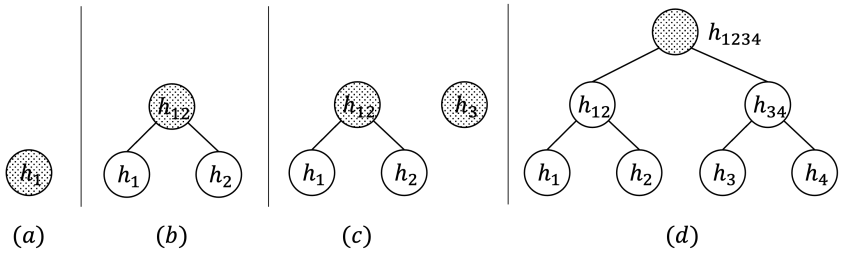


Fig. 2. Changes in snapshot $\langle [hashes], treeSize \rangle$ with the growth of the Merkle tree (Dynamic addition to Merkle tree is adopted from [1]). (a) single node, representation: $\langle [h_1], 1 \rangle$, (b) new node with hash value h_2 is paired with h_1 to compute the hash value h_{12}, representation: $\langle [h_{12}], 2 \rangle$, (c)$h_3$ entered, but adding it does not give us a complete binary tree, so it is put separately, as a complete binary tree by itself, representation: $\langle [h_{12}, h_3], 3 \rangle$, (d) h_4 entered, representation: $\langle [h_{1234}], 4 \rangle$.

Algorithm 1 gives the procedure for inserting new credentials into the tree.

Algorithm 1 Merkle tree `INSERT` algorithm (adapted from [1])

procedure INSERT(MT, x)
  $h \leftarrow MT.hashes$
  $l \leftarrow length(MT.hashes)$ ▷ Size of the array
  $index \leftarrow MT.treeSize + 1$
  $leaf \leftarrow H(index|x)$
  for $i = MT.treeSize$; $i\ 2 = 1$; $i >> 1$ **do** ▷ $>>$ denotes bit shift to the right
    $leaf = H(h[l-1]||leaf)$ ▷ $||$ denotes concatenation
    $l = l - 1$
  end for
  $MT'.treeSize = MT.treeSize + 1$
  $MT'.hashes = MT.hashes[0 : l]$ ▷ $A[0 : l]$ is a subset of A with elements $A[0]$ to $A[l-1]$
  $MT'.hashes = append(MT'.hashes, leaf)$
  return $MT', index$
end procedure

Algorithm 2 Computing Merkle Root (adapted from [1])

```
1: procedure ROOT(MT)
2:     len = length(MT.hashes)
3:     if len = 0 then
4:         return EMPTY
5:     end if
6:     root = MT.hashes[len − 1]
7:     for i = len − 2; i ≥ 0; i − − do
8:         root = H(MT.hashes[i]||root)
9:     end for
10:    return root
11: end procedure
```

$\mathbb{P}\circ\mathbb{C}\$$ **construction.** The $\mathbb{P}\circ\mathbb{C}\$$ construction relies on the existence of a secure credential scheme $\mathbb{CS}$, which must satisfy the *unforgeability* and *non-transferability* properties as described in Section 2. Rather than presenting a specific construction of $\mathbb{CS}$, we adopt a modular approach, leveraging the interfaces of the secure $\mathbb{CS}$. This modular design allows any secure $\mathbb{CS}$ meeting the specified properties to be seamlessly incorporated into the $\mathbb{P}\circ\mathbb{C}\$$ construction. In Section 5, we will exemplify this concept by constructing a secure Proof-of-Location-Subsequence ($\mathbb{P}\circ\mathbb{L}\$$) scheme, which is achieved by integrating a secure Proof-of-Location credential scheme ($\mathbb{P}\circ\mathbb{L}$) into the $\mathbb{P}\circ\mathbb{C}\$$ construction.

1. $\texttt{Init}(1^k) \rightarrow (sk_I, pk_I)$. TA generates a keypair for the issuer I's digital signature $\mathbb{DS}$: $\mathbb{D}\$.KeyGen(1^k) \rightarrow (pk_I, sk_I)$.
2. $\texttt{Issue}(sk_U, pk_I, sk_I, m) \rightarrow cred$ **or** $\perp$**.**
   –(Stage 1) $\texttt{Gen}(pk_U, sk_U, pk_I, sk_I) \rightarrow x$: U sends a message $m \in \{0,1\}^k$ in a credential issue request to I, and I generates credential $x = (m, \sigma)$, where $\sigma \leftarrow \mathbb{D}\$.Sign(sk_I, m)$, and sends x to both U and SM.
   –(Stage 2) $\texttt{Store}(x, pk_I)$: Once the credential x is received from I, U and SM do as follows: (i) U: $\{0,1\} \leftarrow \mathbb{D}\$.Verify\ (pk_I, \sigma, m)$, abort if 0 is the output. Appends x to an ordered list L_{cred}. (ii) SM: $\{0,1\} \leftarrow \mathbb{D}\$.Verify\ (pk_I, \sigma, m)$, abort if 0 is output. If the queue Q is full, calls $Q.Dequeue()$. Lastly, calls $Q.Enqueue(x)$.
3. $\texttt{Update}(Q, s') \rightarrow s$ **or** $\perp$**.** Once the buffer Q at SM is full (i.e., n credentials in Q), SM does the following:[4]
   –(i) Call $DL.read()$ to receive the current snapshot s'.
   –(ii) Retrieve MT' from the snapshot s'.
   –(iii) Run $x \leftarrow Q.Dequeue()$, then run $(MT, index) \leftarrow \texttt{INSERT}(MT', x)$ (see Alg. 1). Append $(x, index)$ to an ordered list L_{insert}, and set $s \leftarrow MT$.
   –(iv) Run step (iii) until all the credentials are removed from Q.

[4] Based on application design requirement, the `Update` operation can be triggered with different events, for example, as soon as a new credential is received by SM, or on the request of the user.

–(v) Send (L_{insert}, s) to U. U, for each $(x, index)$ pair extracted in order from L_{insert}, appends x to an ordered list L_{scred}, append $index$ to an ordered list L_{sindex}.

4. $\texttt{ProofGen}(\mathbf{x}, \mathbf{y}) \rightarrow \pi$**.** U generates a proof π claiming that a credential sequence $\mathbf{y}$ is a subsequence of the credential sequence $\mathbf{x}$. Here, $\mathbf{x}$ essentially consists of all elements of the ordered list L_{scred} in the same order. For each credential $y \in \mathbf{y}$, U performs the following:
   –(i) Finds the associated index $index_y$ from the list L_{sindex}.
   –(ii) Computes a Merkle proof for this credential (see Sec. 2): $\mathbb{MT}.\texttt{GenerateProof}(index_y, R) \rightarrow p_y$. R is the root of the Merkle tree generated for the credential sequence $\mathbf{x}$.
   –(iii) Append $index_y$ to a list L_{CIndex}, and p_y to a list L_{MProof}. Outputs proof $\pi = \{L_{CIndex}, L_{MProof}\}$.
5. $\texttt{Verify}(\pi, \mathbf{y}, s, pk_I) \rightarrow b$**.** U sends to V snapshot s, a credential sequence $\mathbf{y}$, and a proof $\pi = \{L_{CIndex}, L_{MProof}\}$. V performs the following.
   –(i) Calls $DL.read()$ to check if s is the valid snapshot.
   –(ii) Computes the root of the Merkle tree by running $R \leftarrow Root(s)$ (Alg. 2).
   –(iii) For each proof $p_i \in L_{MProof}$ and associated index $i \in L_{CIndex}$ for a credential $y_i \in \mathbf{y}$, runs $\mathbb{MT}.\texttt{VerifyProof}(i||y_i, p_i, R) \rightarrow \{0, 1\}$ (see Sec. 2).
   –(iv) Verifies the signature of the credential issuer for each credential $y_i \in \mathbf{y}$: $\mathbb{DS}.Verify(pk_I, y_i) \rightarrow \{0, 1\}$.
   –(v) If all the above checks succeed, outputs 1, otherwise outputs 0.

Batch verification $\mathbb{PoCS}+$. In the above $\mathbb{PoCS}$ the size of the proof π grows linearly with the number of credentials in the claimed subsequence $\mathbf{y}$. To reduce the proof size of a credential sequence, we *batch* a group of t leaves into a single element of a group that is represented by a binary string, and build the Merkle tree over these strings. Batching uses an efficient polynomial commitment scheme (see Sec. 2) that allows SM to commit to a sequence of t leaves using a single string. SM first generates a polynomial commitment value cmt for a sequence of t data elements $\{\langle i_1, x_1\rangle, ..., \langle i_t, x_t\rangle\}$, and then computes the leaf node as $H(cmt)$. This grouping allows a single string to be used for all credentials in the sequence.

That is for a subsequence $\mathbf{y}$ of credentials in a claimed credential sequence $\mathbf{x}$, which are all within the same batch with commitment value cmt, the user will use a single batch witness value w_B to prove positions of the credentials, and a single Merkle proof for $H(cmt)$ to prove that all credentials in $\mathbf{y}$ are in order. Below we present $\mathbb{PoCS}+$.

1. $\texttt{Init}(1^k, t) \rightarrow (sk_I, pk_I, pk_c, sk_c)$. In addition to the outputs generated in the original scheme, a keypair for the polynomial commitment scheme is generated as $\mathbb{PC}.\texttt{Setup}(1^k, t) \rightarrow (pk_c, sk_c)$. t is an additional input to $\texttt{Init}$ representing the degree of the polynomial, which is also an implicit input to the rest of the algorithms.

2. $\texttt{Issue}(sk_U, sk_I, pk_I, m) \rightarrow cred$ **or** $\perp$**.** This is same as in the original scheme.
3. $\texttt{Update}(Q, s', pk_c) \rightarrow s$ **or** $\perp$**.** Once the buffer size at SM reaches $t+1$ (i.e., $|Q| = t+1$), SM performs following:
–Steps (i) and (ii) performed by SM are the same as in the original scheme. The rest of the steps are modified as below.
–(iii) Run $x_i \leftarrow Q.Dequeue()$ for $i = 1...t+1$. Let the sequence of $t+1$ credentials be $\mathbf{x}$. Create a polynomial $\phi(\mathbf{x})$ of degree t by interpolating the set of data points (Cartesian coordinates) $\{(1, H(x_1)), \ldots, (t+1, H(x_{t+1}))\}$.
–(iv) Run $\mathbb{PC}.\texttt{Commit}(pk_c, \phi(x)) \rightarrow cmt$ to generate polynomial commitment cmt for the polynomial $\phi(\mathbf{x})$.
–(v) Run $(MT, index) \leftarrow \texttt{INSERT}(MT', cmt)$ (see Alg. 1).
–(vi) Append $(\mathbf{x}, cmt, index)$ to an ordered list L_{insert}, and set $s \leftarrow MT$.
–(vii) Send (L_{insert}, s) to U. U, for each $(\mathbf{x}, cmt, index)$ tuple extracted in order from L_{insert}, appends $(\mathbf{x}, cmt)$ pair to an ordered list L_{scred}, append $index$ to an ordered list L_{sindex}.
4. $\texttt{ProofGenerate}(\mathbf{x}, \mathbf{y}) \rightarrow \pi$**.** User U performs the following:
–(i) Group the credentials in $\mathbf{y}$ according to the polynomial that they belong to, with the help of the list L_{scred}. Let, the set of polynomials $\{\phi(x)_1, ..., \phi(x)_q\}$ represent these groups, the set of commitments $\{cmt_1, ..., cmt_q\}$ represent these polynomials, $\{index_1, ..., index_q\}$ represent the respective indexes from the list L_{sindex}. For each group, let the set B_i $(i = \{1, ..., q\})$ consist of the positions of the credentials in the polynomial.
–(ii) For each set B_i, run a batch witness creation algorithm of the polynomial commitment scheme $\mathbb{PC}.\texttt{CreateWitnessBatch}(pk_c, \phi(x)_i, B_i) \rightarrow r(x)_i, w_{B_i}$. Appends $(r(x)_i, B_i, cmt_i)$ into a list L_{CIndex}, cmt_i being the commitment representing the respective group of credentials.
–(iii) Compute a Merkle proof for each commitment cmt_i by running $p_i \leftarrow \mathbb{MT}.\texttt{GenerateProof}\ (index_i, \mathcal{C})$, where $\mathcal{C}$ is the ordered set of all commitments in the list L_{scred}. Append and (w_{B_i}, p_i) into a list L_{MProof}. Outputs proof $\pi = \{L_{CIndex}, L_{MProof}\}$.
5. $\texttt{Verify}(sk_U, \pi, \mathbf{y}, s, pk_I) \rightarrow b$**.** U sends to V snapshot s, a credential sequence $\mathbf{y}$, and a proof $\pi = \{L_{CIndex}, L_{MProof}\}$. V performs the following.
–(i) Calls $DL.read()$ to check if s is the valid snapshot.
–(ii) Computes the root of the Merkle tree by running $R \leftarrow Root(s)$ (Alg. 2).
–(iii) For each proof $p_i \in L_{MProof}$ and associated index $i \in L_{CIndex}$ for the commitment cmt_i, runs $\mathbb{MT}.\texttt{VerifyProof}(i||cmt_i, p_i, R) \rightarrow \{0, 1\}$ (see Sec. 2).
–(iv) for each witness w_{B_i}, run $\mathbb{PC}.\texttt{VerifyEvalBatch}(pk_c, cmt, B, r(x)_i, w_{B_i}) \rightarrow \{1, 0\}$.
–(v) Based on the last two steps, verify the claimed position of each credential $y_i \in \mathbf{y}$.
–(vi) Verifies the signature of the credential issuer for each credential $y_i \in \mathbf{y}$: $\mathbb{DS}.Verify(pk_I, y_i) \rightarrow \{0, 1\}$.
–(vii) If the above checks succeed, output 1, otherwise output 0.

Efficiency comparison. To verify a subsequence of n credentials, the number of Merkle proofs required is n in the original $\mathbb{P}\circledcirc\mathbb{C}\$$ scheme. In the $\mathbb{P}\circledcirc\mathbb{C}\$+$ scheme, this number is 1 in the best case, given that all n credentials are from the same polynomial. In the worst case, where each of the n credentials is from a different polynomial, this number is $N/(t+1)$, N being the number of total credentials issued to the user, and t is the degree of the polynomial.

Theorem 1. *Proposed* $\mathbb{P}\circledcirc\mathbb{C}\$$ *and* $\mathbb{P}\circledcirc\mathbb{C}\$+$ *schemes achieve (i) subsequence membership (Property 1) assuming* H *to be a collusion-resistant hash function, the digital signature scheme* DS *to be resistant against existential forgery, the polynomial commitment scheme* $\mathbb{PC}$ *to be computationally binding (*$\mathbb{P}\circledcirc\mathbb{C}\$+$*) and that all the credentials are received by both the user and the snapshot manager instantaneously, or with a fixed delay, and immediately after generation by the credential issuer. (ii) selective reveal (Property 2) assuming* H *to be preimage resistant and* $\mathbb{PC}$ *to be computationally hiding (*$\mathbb{P}\circledcirc\mathbb{C}\$+$*).*

The proof is given in Appendix B.

4 Proof-of-Location ($\mathbb{P}\circledcirc\mathbb{L}$)

We start with the model and construction of a Proof-of-Location ($\mathbb{P}\circledcirc\mathbb{L}$) scheme that provides user anonymity against the verifier, then use it to propose a Proof-of-Location-Subsequence ($\mathbb{P}\circledcirc\mathbb{L}\$$) scheme in the next section.

A $\mathbb{P}\circledcirc\mathbb{L}$ scheme enables a user U to obtain a proof-of-location (*pol*) token from an issuer I at a geo-location ℓoc, that they can present to a verifier V as evidence of visiting the location ℓoc. The scheme has four entities. A trusted authority (TA) generates system parameters and registers users into the system. A set of users $\mathcal{U}$ that collect proof-of-location (*pol*) tokens from issuers, and present these to the verifiers. A set of issuers $\mathcal{I}$ that verify users' locations and issues proof-of-location (*pol*) tokens, and a set of verifiers $\mathcal{V}$ that verify *pol* tokens presented by users.

Trust assumptions. Users are dishonest and may attempt to forge or falsely claim ownership of a *pol* token. Verifiers can be dishonest and attempt to link *pol* tokens to user identities, to deduce the travel trajectory of the user. Issuers are considered trusted who follow the protocol correctly and do not help the verifiers to link *pol* tokens to the user identities. Each user and issuer has a geo-location $\ell oc = (x, y) \in \mathbb{R} \times \mathbb{R}$ assigned to it.

The $\mathbb{P}\circledcirc\mathbb{L}$ scheme consists of four algorithms.

- $\texttt{PoLSetup}(1^k) \rightarrow par_{\mathbb{P}\circledcirc\mathbb{L}}$. TA generates public and private parameters of the system. This includes signature keypair (sk_{TA}, pk_{TA}) for the TA, keypair (sk_I, pk_I) for each issuer $I \in \mathcal{I}$.
- $\texttt{PoLRegister}(sk_U, sk_{TA}) \rightarrow Cred_U$. U is registered into the system and credential $Cred_U$ is generated for the user by TA, on input user secret key sk_U and TA's secret key sk_{TA}.

- $\texttt{PoLIssue}(Cred_U, sk_I) \rightarrow pol$. I verifies U's proximity, and generates a proof-of-location token pol.
- $\texttt{PoLVerify}(Cred_U, pol, pk_I) \rightarrow b$. V verifies that pol is issued by a valid issuer to the presenter of the token, and outputs $b = 1$ (accept) or $b = 0$ (reject).

We follow the security model of [4] to define three properties for $\mathbb{P}\circledcirc\mathbb{L}$: *(i) Unforgeability:* A *pol* token that is successfully verified by V must have been issued by a I, and the user was within a given distance at the time of *pol* generation. *(ii) Non-transferability: A pol token verified successfully for user U, must not have been issued to a different user U'. (iii) Anonymity:* Verifier must not be able to link a verification session to a user.

4.1 $\mathbb{P}\circledcirc\mathbb{L}$ Construction

Below we provide a construction for the $\mathbb{P}\circledcirc\mathbb{L}$ scheme. For user proximity verification, we use the distance bounding protocol proposed in [4].

1. $\texttt{PoLSetup}(1^k) \rightarrow par_{\mathbb{P}\circledcirc\mathbb{L}}$. TA computes $par_{\mathbb{P}\circledcirc\mathbb{L}}$, which include:
   –(i) signature keypair: $\mathbb{DS}.\texttt{KeyGen}(1^k) \rightarrow pk_{TA}, sk_{TA}$,
   –(ii) common parameter of the designated verifier signature scheme: $\mathbb{DVS}.\texttt{GenCP}(1^k) \rightarrow cp$,
   –(iii) For each issuer $I \in \mathcal{I}$, generate public key encryption keypair: $\mathbb{E}.\texttt{KeyGen}(1^k) \rightarrow pk_I^e, sk_I^e$,
   –(iv) For each issuer $I \in \mathcal{I}$, generate signer key-pair of the designated verifier signature scheme: $\mathbb{DVS}.\texttt{GenSKey}(cp) \rightarrow (sk_I, pk_I)$,
   –(v) For each verifier $V \in \mathcal{V}$, generate verifier keypair of the designated verifier signature scheme: $\mathbb{DVS}.\texttt{GenVKey}(cp) \rightarrow (sk_V, pk_V)$, and runs the key-registration protocol of the designated verifier signature scheme: $\mathbb{DVS}.\texttt{KeyRegV}(cp) \rightarrow (pk_V, \{0,1\})$,
   –(vi) Generate public key of the commitment scheme: $\mathbb{C}.\texttt{KeyGen} \rightarrow PK_c$.
   Public parameters in $par_{\mathbb{P}\circledcirc\mathbb{L}}$ are implicit inputs to the rest of the algorithms.
2. $\texttt{PoLRegister}(sk_U, sk_{TA}) \rightarrow Cred_U$.
   – (i) U computes $\mathbb{DS}.\texttt{KeyGen}(1^k) \rightarrow pk_U, sk_U$ and ends pk_U to TA.
   –(ii) U and TA run the protocol for signing a committed value [11], outputting $Cert_U \leftarrow \mathbb{DS}.\texttt{Sign}(sk_{TA}, sk_U)$.
   –(iii) U: stores $Cred_U = (sk_U, Cert_U)$.
3. $\texttt{PoLIssue}(Cred_U, sk_I) \rightarrow pol$.
   –(i) U and I run a distance bounding (DB) protocol as presented in [4].
   –(ii) If the DB protocol outputs 1, I runs the $\texttt{Sig}$ algorithm of the DV signature scheme on a message that contains the committed value cmt (i.e., committed by the user on its secret sk_U), I's public key pk_I and I's location ℓoc_I. *pol* token, is sent to U. $pol = \langle msg, \sigma \rangle$ $\sigma = \mathbb{DVS}.\texttt{Sig}(sk_I, msg)$, $msg = \langle cmt, pk_I, \ell oc_I \rangle$
4. $\texttt{PoLVerify}(Cred_U, pol, pk_I) \rightarrow b$.
   –(i) U generates a designated verifier (DV) signature: $\mathbb{DVS}.\texttt{Des}$ (pk_I, pk_V,

$msg, \sigma) \rightarrow \hat{\sigma}$. Send $\hat{pol} = (msg, \hat{\sigma})$ to V, encrypted with V's public key pk_V. –(ii) V decrypts the values and runs the designated verification algorithm $\mathbb{DVS}.\texttt{DVer}(pk_I, sk_V, msg, \hat{\sigma}) \rightarrow \{0, 1\}$. Rejects and aborts if the output is 0. –(iii) V and U run the $\mathbb{ZKPK}$ protocol in [11] to verify that the presenter U has the knowledge of the secret key sk_U that was committed in cmt, and also knows the certificate on the secret generated by TA. V outputs 1 (accept) or 0 (reject) accordingly.

Theorem 2. *Assuming $\mathbb{E}$ to be IND-CCA secure, $\mathbb{C}$ to be computationally binding and computationally hiding, the digital signature scheme $\mathbb{DS}$ and the designated verifier signature scheme $\mathbb{DVS}$ to provide existential unforgeability, $\mathbb{ZKPK}$ is sound and zero-knowledge proof-of-knowledge of the values $(sk_U, a, Cert_U)$, then, (i) Assuming DB is secure, $\mathbb{P}\circ\mathbb{L}$ is unforgeable. (ii) Assuming the user is not willing to share their secret credential, $\mathbb{P}\circ\mathbb{L}$ provides non-transferability. (iii) $\mathbb{P}\circ\mathbb{L}$ is anonymous with respect to the verifier.*

The proof is in the Appendix B.

5 Proof-of-Location-Subsequence ($\mathbb{P}\circ\mathbb{L}\$$)

A $\mathbb{P}\circ\mathbb{L}\$$ scheme, as the $\mathbb{P}\circ\mathbb{L}$ scheme (Section 4), has a trusted authority (TA), a set of users $\mathcal{U}$, a set of issuers $\mathcal{I}$, a set of verifiers $\mathcal{V}$. Additionally, it has a snapshot manager SM and a public distributed ledger DL, as in the $\mathbb{P}\circ\mathbb{C}\$$ scheme.

Definition 4 (Proof-of-Location-Subsequence $\mathbb{P}\circ\mathbb{L}\$$). *$\mathbb{P}\circ\mathbb{L}\$$ has six algorithms:*

1. $\texttt{Init}(1^k) \rightarrow par_{\mathbb{P}\circ\mathbb{L}}, par_{\mathbb{P}\circ\mathbb{L}\$}$*: TA runs* $\texttt{PoLSetup}(1^k) \rightarrow par_{\mathbb{P}\circ\mathbb{L}}$*, and some additional parameters called $par_{\mathbb{P}\circ\mathbb{L}\$}$ for the system.*
2. $\texttt{Register}(sk_U, sk_{TA}) \rightarrow Cred_U$*: This is the* $\texttt{PoLRegister}$ *protocol of the $\mathbb{P}\circ\mathbb{L}$ scheme.*
3. $\texttt{Issue}(Cred_U, sk_I) \rightarrow pol$*: (i) U and I run the* $\texttt{PoLIssue}(Cred_U, sk_I, pk_I) \rightarrow pol$ *protocol of the $\mathbb{P}\circ\mathbb{L}$ scheme. In addition to user U, I sends the generated pol to the snapshot manager SM. (ii) U and SM runs $\mathbb{P}\circ\mathbb{C}\$.\texttt{Store}(pol, pk_I)$ algorithm.*
4. $\texttt{Update}(Q, s') \rightarrow s$*: Once the buffer Q in SM is full, SM runs the* $\texttt{Update}(Q, s') \rightarrow s$ *protocol of the $\mathbb{P}\circ\mathbb{C}\$$ scheme. Instead of the plain snapshot s, a function of the snapshot $f(s)$ is written on the distributed ledger DL by SM.*
5. $\texttt{ProofGen}(\mathbf{x}, \mathbf{y}) \rightarrow \pi$*: U runs the* $\texttt{ProofGen}(\mathbf{x}, \mathbf{y}) \rightarrow \pi$ *algorithm of the $\mathbb{P}\circ\mathbb{C}\$$ scheme, where $\mathbf{x} = \{pol_1^x, ..., pol_n^x\}$ and $\mathbf{y} = \{pol_1^y, ..., pol_m^y\}$ are two sets of pol tokens.*
6. $\texttt{Verify}(\pi, \mathbf{y}, s, pk_I) \rightarrow b$*: (i) For each $pol \in \mathbf{y}$, U and V run* $\texttt{PoLVerify}$ $(Cred_U, pol, pk_I) \rightarrow b$ *of the $\mathbb{P}\circ\mathbb{L}$ scheme. If $b = 0$ is returned for any of these protocol runs, the verifier rejects and aborts with output 0. (ii) U and V runs* $\texttt{Verify}(\pi, \mathbf{y}, s, pk_I) \rightarrow b$ *protocol of the $\mathbb{P}\circ\mathbb{C}\$$ scheme. If $b = 0$, the verifier rejects and aborts with output 0. Otherwise, it outputs 1.*

Security game. As in $\mathbb{P}\text{o}\mathbb{C}\$$, we take a game-based approach to define the security of the $\mathbb{P}\text{o}\mathbb{L}\$$ scheme where A has access to the four types of oracle queries: i. $Corrupt(P)$, ii. $Issue(U, I)$ to run $\mathbb{P}\text{o}\mathbb{L}\$$.`Issue` protocol between U and I. iii. $Update(\mathbf{x}, s', U)$ and to run $\mathbb{P}\text{o}\mathbb{L}\$$.`Update` protocol between U and SM. iv. $Verify(\pi, s, \mathbf{y}, U, V)$ to run $\mathbb{P}\text{o}\mathbb{L}\$$.`Verify` protocol between U and V.

Definition 5 ($\mathbb{P}\text{o}\mathbb{L}\$$ Game). *For a security parameter k, a game between the challenger C and the adversary A has three steps:*

1. *Initialize: C runs* `Setup` *to generate system parameters. C also initializes lists $L_{corrupt}$, L_{issue}, L_{update}, and L_{verify}.*
2. *Generate entities:*
   (a) *C generates a set of users $\mathcal{U}$, a set of issuers $\mathcal{I}$, a set of verifiers $\mathcal{V}$.*
   (b) *C credentials (public/private key pairs) for the issuers and the verifiers and sets arbitrary locations for the users and issuers.*
   (c) *C runs* `Register` *for each user $U \in \mathcal{U}$.*
   (d) *Lastly, C publishes the list $\mathcal{U}$, $\mathcal{I}$, $\mathcal{V}$, and their public credentials and locations.*
3. *Queries: Adversary makes a polynomial number of oracle queries listed above.*
4. *Adversary outputs: Two sequences of pol tokens $\mathbf{x}, \mathbf{y}$ (in $\mathbb{P}\text{o}\mathbb{L}\$$ Integrity game) or a bit $\hat{b}$ (in $\mathbb{P}\text{o}\mathbb{L}\$$ Unlinkability game).*

Properties. We define two properties for $\mathbb{P}\text{o}\mathbb{L}\$$: (i) Integrity: Addressing the forgery and token-transfer attacks on the proof-of-location token *pol*, as well as the attacks involving forging a *pol* into the $\mathbb{P}\text{o}\mathbb{L}\$$ claim, and tampering with the issuing order of *pol* tokens. (ii) Unlinkability: verifier cannot link two tokens $pol_1, pol_2, pol_1 \neq pol_2$ from two different verification sessions, such that they belong to the same user. We allow collusion among verifiers. The only assumption we make is that, if both verification sessions are with the same verifier, pol_1, pol_2 are not from two "overlapping" verification sessions. Two verification sessions are said to be overlapping if the two sets of *pol* tokens presented in the two sessions have common *pol*s. However, we do allow overlapping verification sessions run by two different verifiers, even if they are colluding.

Property 3 ($\mathbb{P}\text{o}\mathbb{L}\$$-Integrity). Consider a proof-of-location subsequence scheme $\mathbb{P}\text{o}\mathbb{L}\$$ and a $\mathbb{P}\text{o}\mathbb{L}\$$ Game. A can only corrupt users. A outputs two sets of *pol* tokens, $\mathbf{x} = \{pol_1^x, \ldots, pol_m^x\}$ and $\mathbf{y} = \{pol_1^y, \ldots, pol_n^y\}$, with $m \geq n$. Adversary wins the game if there exists an entry $(U, ., ., \mathbf{y}, ., ., b = 1) \in L_{verify}$, and at least one of the following conditions is satisfied:

1. There exists a $pol \in \mathbf{y}$ such that it was not generated or issued by a trusted issuer, i.e., $(pol, ., .) \notin L_{issue}$.
2. There exists a $pol \in \mathbf{y}$ such that it was issued to a different user, i.e., $(pol, U', .) \in L_{issue}$, $U' \neq U$.
3. There exists a $pol \in \mathbf{y}$ such that it was not input of any `Update` protocol run, i.e., $(., ., pol, .) \notin L_{update}$.
4. There exists a pair $(pol_i^y, pol_j^y) \in \mathbf{y}$ such that $i < j$, and a pair $(pol_{i'}^x, pol_{j'}^x) \in \mathbf{x}$ such that $pol_i^y = pol_{i'}^x$, $pol_{j'}^y = pol_j'^x$, and $i' > j'$.

A $\mathbb{P}\circ\mathbb{L}\$$ scheme achieves integrity if the advantage of the adversary to win this game is negligible.

Property 4 ($\mathbb{P}\circ\mathbb{L}\$$-Unlinkability). Consider a $\mathbb{P}\circ\mathbb{L}\$$ scheme and a $\mathbb{P}\circ\mathbb{L}\$$ game. A can only corrupt verifiers. We replace steps iii and iv of the game with the following steps:

1. A sends following choices to C: a pair of users $(U_0, U_1) \in \mathcal{U}$, a pair of verifiers $(V_0, V_1) \in \mathcal{V}$. Adversary is allowed to set $V_1 = V_0$, an array of n locations $Loc_X = [\ell oc_1, \ldots, \ell oc_n]$, and Loc_Y, Loc_Z, two subsequenes of Loc_X.
2. If $V_0 = V_1$ and $Loc_Y \cap Loc_Z \neq \{\Phi\}$, C rejects and abort.
3. For each $\ell oc_i \in Loc_X$, C does the following:
   (a) Selects an issuer $I \in \mathcal{I}$ with $\ell oc_I = \ell oc_i$. If no such issuer, create one and set its location as loc_i.
   (b) Chooses a random bit $b \leftarrow \{0, 1\}$.
   (c) Runs $Issue(U_b, I) \rightarrow pol$, followed by $Update(\{pol\}, s, U_b) \rightarrow s'$.
   (d) Repeats the last step for user U_{1-b}.
4. C picks two independent random bits $b_0, b_1 \leftarrow \{0, 1\}$.
5. C runs $Verify$ oracle twice: 1. $Verify$ $(\pi_{b_0}, \mathbf{y}, s_{b_0}, U_{b_0}, V_0)$. $\mathbf{y}$ is the sequence of pols generated at the location subsequence Loc_Y. 2. $Verify$ $(\pi_{b_1}, Z, s_{b_1}, U_{b_1}, V_1)$. Z is the sequence of pols generated at the location subsequence Loc_Z.
6. At the end of the $\mathbb{P}\circ\mathbb{L}\$$-game, A outputs a bit $\hat{b} \in \{0, 1\}$, , 0 denoting the statement "both verification sessions belong to the same user", and 1 denoting the statement "verification sessions belong to different users".

A wins the game if $\hat{b} = b_0 \oplus b_1$. The $\mathbb{P}\circ\mathbb{L}\$$ scheme satisfies $\mathbb{P}\circ\mathbb{L}\$$-Unlinkability if the adversary's advantage in this game is negligible.

5.1 $\mathbb{P}\circ\mathbb{L}\$$ construction

In addition to the cryptographic primitives used in $\mathbb{P}\circ\mathbb{L}$ and $\mathbb{P}\circ\mathbb{C}\$$, the $\mathbb{P}\circ\mathbb{L}\$$ construction uses the accumulator ($\mathbb{A}$).

1. $\texttt{Init}(1^k) \rightarrow par_{\mathbb{P}\circ\mathbb{L}}, par_{\mathbb{P}\circ\mathbb{L}\$}$**:** TA runs $\mathbb{P}\circ\mathbb{L}$.`Setup`, and generates public parameters for the accumulator $\mathbb{A}$: $\texttt{SetupAcc}(1^k) \rightarrow par_A$.
2. $\texttt{Register}(sk_U, sk_{TA}) \rightarrow Cred_U$**:** U and TA runs `PoLRegister` of $\mathbb{P}\circ\mathbb{L}$ scheme.
3. $\texttt{Issue}(Cred_U, sk_I) \rightarrow pol$**:** U and I runs `PoLIssue` of the $\mathbb{P}\circ\mathbb{L}$ scheme. In the last step of the protocol, I sends pol to U and SM simultaneously. Then U and SM run $\mathbb{P}\circ\mathbb{C}\$$.$\texttt{Store}(pol, pk_I)$ algorithm.
4. $\texttt{Update}(Q, s') \rightarrow s$**:** We assume that SM keeps a mapping M between each user $U \in \mathcal{U}$ and their respective snapshot s. For a user U, once the buffer at SM is full (original $\mathbb{P}\circ\mathbb{C}\$$) or reaches $(t+1)$ pols ($\mathbb{P}\circ\mathbb{C}\$+$), SM does the following:
   -(i) Runs $\mathbb{P}\circ\mathbb{C}\$$.$\texttt{Update}(Q, s') \rightarrow s$.
   -(ii) Updates the mapping M with new snapshot s.

-(iii) Either initiates an accumulator value by running $\texttt{Acc}(par_A, \{s\})$ (if this is the first update request from any user in the system), or updates the accumulator value Acc' that is stored on DL, by running $\texttt{UpdateAcc}(par_A, L, s', s) \rightarrow Acc$, L is the list of all snapshots accumulated in Acc', s', s being the old and new snapshot value for user U. Updated Acc is written on DL.
-(iv) Runs $\texttt{GenWit}(par_A, s, L) \rightarrow w$ to generate witness w for this new accumulator and sends it to all users in $\mathcal{U}$.

5. $\texttt{ProofGen}(\mathbf{x}, \mathbf{y}) \rightarrow \pi$**:** U runs $\mathbb{P}\circ\mathbb{C}\mathbb{S}.\texttt{ProofGen}(\mathbf{x}, \mathbf{y}) \rightarrow \pi$, $\mathbf{x}$ and $\mathbf{y}$ being the sets of *pol* tokens.
6. $\texttt{Verify}(\pi, \mathbf{y}, s, pk_I) \rightarrow b$**:**
-(i) For each $pol \in \mathbf{y}$, U and V runs $\mathbb{P}\circ\mathbb{L}.\texttt{PoLVerify}\ (Cred_U, pol, pk_I) \rightarrow b$.
-(ii) U and V runs $\mathbb{P}\circ\mathbb{C}\mathbb{S}.\texttt{Verify}(\pi, \mathbf{y}, s, pk_I) \rightarrow b$.
-(iii) V calls $DL.read()$ to read the accumulator value Acc on the DL, then runs a membership verification [10] with U, verifying that s is a member of Acc, without knowing the witness w.

Theorem 3. *Assuming* $\mathbb{P}\circ\mathbb{C}\mathbb{S}$ *satisfies subsequence membership and selective reveal,* $\mathbb{P}\circ\mathbb{L}$ *satisfies unforgeability, non-transferability, and anonymity, the accumulator* $\mathbb{A}$ *satisfies strong collision-resistance,* $\mathbb{DVS}$ *satisfies non-transferability privacy, the proposed* $\mathbb{P}\circ\mathbb{L}\mathbb{S}$ *scheme achieves integrity and unlinkability.*

The proof is in the Appendix B.

6 KROM feasibility for blockchain integration

KROM utilizes a distributed ledger (DL) to store the accumulated value of user snapshots, which needs to be updated by the snapshot manager (SM) whenever a user sends a snapshot update request. During verification, the verifier also needs to access this value.

Among various available public ledgers with storage and lookup features, we opted for NameCoin for our research due to its relatively low operating cost. NameCoin offers a built-in mechanism for storing key-value pairs, where the keys have a namespace as a prefix. It also provides basic functionality to scan existing names. As a result, we can register a name on the blockchain that can only be updated by the snapshot manager (SM), as the key required to update the data value is only known by SM.

To register a name on NameCoin and write data (accumulated value Acc of snapshots), a very small fee needs to be paid (currently 0.01 USD). This fee essentially purchases a name in the system's namespace by associating a public key as the owner of the name using the `name_new` command. Once a name is registered, the value for the name can be updated with the accumulator value using the `name_update` command. However, NameCoin has a limitation of only allowing 520 bytes of storage under a name, whereas the size of an RSA accumulator value is 1024 bytes. This challenge can be overcome by registering two names and storing parts of the accumulator value under these names. Updating

a value for a name costs 0.004 USD. A verifier can retrieve the value of the accumulator by searching for the names that SM has registered on NameCoin using the `name_show` command.

NameCoin aims to create blocks every 10 minutes, but the actual wait time may vary based on network propagation delays and transaction volume. Given this latency constraint, NameCoin is less suitable for $\mathbb{P}\text{o}\mathbb{L}\$$ applications that require the stored accumulator to be updated at a faster rate.

7 Related work

Proof-of-location sequence. Previous works [24, 22] use the term "location provenance" to refer to the history of a user's locations within a specific timeframe. [24] outlines requirements for designing such schemes, including order preservation and selective reveal, and explores techniques like hash chains, block hash chains, and bloom filters to fulfill these requirements. However, they do not provide complete construction of Proof-of-Location-Subsequence. On the other hand, [22] presents a comprehensive scheme but fails to satisfy the selective reveal property. Additionally, [26, 25] propose two other $\mathbb{P}\text{o}\mathbb{L}$ schemes that allow users to prove a subsequence of *pol*s. However, none of these works achieve the unlinkability property or provide a formal security model for their schemes. Our work is the first to enable proof of *pol* subsequences while ensuring the unlinkability of verification sessions.

Credential storage and proof-of-subsequence. Previous work [5] explores blockchain storage of credentials, proposing a $\mathbb{P}\text{o}\mathbb{L}$ scheme where all generated *pol*s are stored on the blockchain. However, this approach lacks privacy for *pol* tokens and requires extensive storage on the chain. Another approach considered in [18] involves storing credentials offline and using the blockchain as an anchor. Unfortunately, none of these approaches enable subsequence verification. Industry solutions such as Sovrin (sovrin.org), Blockstack (blockstack.org), Chainpoint (chainpoint.org), POEX.IO, and Chainy (chainy.info) have also been examined in this context. Only Chainpoint offers subsequence verification, but it comes with a costly update and verification process.

8 Concluding Remarks

We address the problem of efficient credential storage guaranteeing the integrity of the credentials sequence and propose a proof-of-credential-subsequence scheme as a solution that also ensures privacy. Building upon this, we propose a proof-of-location scheme with private subsequence verification, that does not require a synchronized clock. This is the first work that allows an untrusted user to prove a subsequence of *pol* tokens to a verifier while ensuring the unlinkability of verification sessions.

References

1. Ontology technology white paper: A new high-performance public multi-chain project & a distributed trust collaboration platform. Tech. rep., Ontology.io (March 2018), https://ont.io/wp/Ontology-technology-white-paper-EN.pdf
2. Ahmadi, A., et al.: Anonymous Distance-Bounding Identification. Tech. Rep. 365, University of Calgary, AB (2018)
3. Akand, M., Safavi-Naini, R.: In-region authentication. In: ACNS 2018. pp. 557–578. Springer (2018)
4. Akand, M., et al.: Privacy-preserving proof-of-location with security against geo-tampering. IEEE TDSC **20**(1), 131–146 (2023)
5. Amoretti, M., et al.: Blockchain-based proof of location. In: QRS-C 2018. pp. 146–153. IEEE (2018)
6. Avoine, G., et al.: A Terrorist-fraud Resistant and Extractor-free Anonymous Distance-bounding Protocol. In: ASIACCS 2017. pp. 800–814. ACM (Apr 2017)
7. Benaloh, J., Mare, M.D.: One-way accumulators: A decentralized alternative to digital signatures. In: Workshop on the Theory and Application of Cryptographic Techniques. pp. 274–285. Springer (1993)
8. Boneh, D., Boyen, X.: Short signatures without random oracles. In: International conference on the theory and applications of cryptographic techniques. pp. 56–73. Springer (2004)
9. Boureanu, I., et al.: Practical and provably secure distance-bounding. Journal of Computer Security **23**(2), 229–257 (Jan 2015)
10. Camenisch, J., Lysyanskaya, A.: Dynamic accumulators and application to efficient revocation of anonymous credentials. In: CRYPTO. pp. 61–76. Springer (2002)
11. Camenisch, J., Lysyanskaya, A.: A signature scheme with efficient protocols. In: SecureComm. pp. 268–289. Springer (2002)
12. Camenisch, J., Stadler, M.: Efficient group signature schemes for large groups. In: Annual International Cryptology Conference. pp. 410–424. Springer (1997)
13. Chase, M., et al.: The signal private group system and anonymous credentials supporting efficient verifiable encryption. In: ACM CCS. pp. 1445–1459 (2020)
14. Connolly, A., et al.: Protego: Efficient, revocable and auditable anonymous credentials with applications to hyperledger fabric. In: INDOCRYPT 2022. pp. 249–271. Springer (2023)
15. Čučko, Š., Turkanović, M.: Decentralized and self-sovereign identity: Systematic mapping study. IEEE Access **9**, 139009–139027 (2021)
16. Damagard, I., Fujisaki, E.: An integer commitment scheme based on groups with hidden order. IACR Cryptology ePrint Archive 2001/064 (2001)
17. Ferdous, M., et al.: In search of self-sovereign identity leveraging blockchain technology. IEEE access **7**, 103059–103079 (2019)
18. Garman, C., et al.: Decentralized anonymous credentials. Cryptology ePrint Archive (2013)
19. Goldwasser, S., et al.: A digital signature scheme secure against adaptive chosen-message attacks. SIAM Journal on computing **17**(2), 281–308 (1988)
20. Grech, A., et al.: Blockchain, self-sovereign identity and digital credentials: promise versus praxis in education. Frontiers in Blockchain **4**, 616779 (2021)
21. Hanzlik, L., Slamanig, D.: With a little help from my friends: Constructing practical anonymous credentials. In: ACM CCS. pp. 2004–2023 (2021)
22. Hasan, R., et al.: WORAL: A witness oriented secure location provenance framework for mobile devices. IEEE Transactions on Emerging Topics in Computing **4**(1), 128–141 (2015)

23. Kate, A., et al.: Constant-size commitments to polynomials and their applications. In: International conference on the theory and application of cryptology and information security. pp. 177–194. Springer (2010)
24. Khan, R., et al.: OTIT: Towards secure provenance modeling for location proofs. In: Proceedings of the 9th ACM symposium on Information, computer and communications security. pp. 87–98 (2014)
25. Khan, R., et al.: 'who, when, and where?' location proof assertion for mobile devices. In: IFIP Annual Conference on Data and Applications Security and Privacy. pp. 146–162. Springer (2014)
26. Lyu, C., et al.: Clip: Continuous location integrity and provenance for mobile phones. In: IEEE MASS. pp. 172–180. IEEE (2015)
27. Merkle, R.: A digital signature based on a conventional encryption function. In: Conference on the theory and application of cryptographic techniques. pp. 369–378. Springer (1987)
28. Naik, N., Jenkins, P.: Self-sovereign identity specifications: Govern your identity through your digital wallet using blockchain technology. In: MobileCloud. pp. 90–95. IEEE (2020)
29. Pinto, R.: Costs of storing data on the blockchain. https://www.linkedin.com/pulse/costs-storing-data-blockchain-rohan-pinto/ (2 2021), (Accessed on 11/16/2022)
30. Rackoff, C., et al.: Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In: CRYPTO'91. pp. 433–444. Springer (1991)
31. Rackoff, C., Simon, D.R.: Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In: Advances in Cryptology—CRYPTO'91: Proceedings. pp. 433–444. Springer (2001)
32. Rosenberg, M., et al.: zk-creds: Flexible anonymous credentials from zksnarks and existing identity infrastructure. Cryptology ePrint Archive (2022)
33. Steinfeld, R., et al.: Universal designated-verifier signatures. In: Asiacrypt. pp. 523–542. Springer (2003)
34. Tobin, A., Reed, D.: The inevitable rise of self-sovereign identity. The Sovrin Foundation **29**(2016), 18 (2016)

A Additional preliminaries

Encryption. An encryption scheme, denoted as $\mathbb{E}$, comprises three fundamental algorithms: key generation ($\mathbb{E}.\texttt{KeyGen}$), encryption ($\mathbb{E}.\texttt{Enc}$), and decryption ($\mathbb{E}.\texttt{Dec}$). Given a security parameter λ, the execution of these algorithms is defined as follows:

- $\mathbb{E}.\texttt{KeyGen}(1^\lambda)$ outputs a pair of public and private keys, represented as (pk_e, sk_e).
- $\mathbb{E}.\texttt{Enc}(pk_e, m)$ takes the public key pk_e and a message m as input, producing the ciphertext C.
- $\mathbb{E}.\texttt{Dec}(sk_e, C)$ takes the private key sk_e and a ciphertext C as input, and returns the original plaintext message m.

The correctness property mandates that for any key pair generated by $\mathbb{E}.\texttt{KeyGen}$, the decryption ($\mathbb{E}.\texttt{Dec}$) of any message m encrypted with $\mathbb{E}.\texttt{Enc}$ must correctly recover the original plaintext m.

This paper focuses on the standard notion of indistinguishability under chosen ciphertext (IND-CCA) attack [31], which ensures the security of the encryption scheme against adversaries.

Digital signature. A digital signature scheme, denoted as $\mathbb{DS}$, encompasses three essential algorithms: key generation ($\mathbb{DS}.\texttt{KeyGen}$), signature generation ($\mathbb{DS}.\texttt{Sig}$), and signature verification ($\mathbb{DS}.\texttt{Vf}$). With a security parameter λ, the execution of these algorithms is defined as follows:

- $\mathbb{DS}.\texttt{KeyGen}(1^\lambda)$ generates a pair of public and private keys denoted as (pk_s, sk_s).
- $\mathbb{DS}.\texttt{Sig}(sk_s, m)$ takes the signing key sk_s and a message m as input, producing the signature σ.
- $\mathbb{DS}.\texttt{Vf}(pk_s, m, \sigma)$ takes the public verification key pk_s, a message m, and a signature σ as input, and outputs either 0 or 1 indicating the verification result.

The correctness requirement dictates that for any key pair generated by $\mathbb{DS}.\texttt{KeyGen}$ and for any message m signed by $\mathbb{DS}.\texttt{Sig}$, the $\mathbb{DS}.\texttt{Vf}$ algorithm must output 1 to indicate the validity of the signature.

We adopt the standard notion of security against existential forgery in this paper, where an attacker aims to discover a signature σ for at least one new message m that is valid with respect to a given verification key.

The paper utilizes the CL-signature scheme proposed by Camenisch and Lysianskaya [11], which enables users to commit to a message and subsequently prove knowledge of the signature on that message while ensuring message confidentiality. This scheme adheres to the conventional definition of signature schemes established by Goldwasser, Micali, and Rivest [19].

Commitment ($\mathbb{C}$). A commitment protocol ($\mathbb{C}$) is a two-party protocol involving a committer and a receiver. It consists of two stages: the *Commitment* stage and the *Reveal* stage. During the *Commitment* stage, the committer generates a commitment c for a value x. In the subsequent *Reveal* stage, the committer opens the commitment, revealing the value x'.

A commitment scheme is considered *perfectly hiding* if, after the *Commitment* stage, the receiver cannot gain any information about the original committed value x. On the other hand, it is *perfectly binding* if, during the *Reveal* stage, the committer can only open the commitment to the initially committed value x (i.e., $x' = x$). The formal definitions of these properties can be found in [16]. These properties can be satisfied against an unbounded adversary, resulting in statistical security, or against a polynomially bounded adversary, resulting in computational security.

In this paper, we employ a commitment scheme proposed by Damgard and Fujisaki [16]. For a security parameter λ, the $\texttt{KeyGen}$ algorithm generates a public key $PK_c = (n, g, h)$, where n is a special RSA modulus. The values $h \leftarrow QR_n$ and $g \leftarrow \langle h \rangle$ are chosen, with $\langle h \rangle$ representing the group generated by h, and QR_n denoting the quadratic residues modulo n. The commitment

$com = \texttt{Commit}(x, r)$ for a string x is computed using a randomly selected string $r \in \mathbb{Z}_n$ as $com = g^x \times h^r \mod n$. During the reveal stage, the committer discloses the values x and r, allowing the receiver to verify $com = \texttt{Commit}(x, r)$.

This commitment scheme is *statistically hiding* and *computationally binding*, assuming the hardness of the factoring problem.

Zero-knowledge proof of knowledge ($\mathbb{ZKPK}$). Zero-knowledge proof of knowledge ($\mathbb{ZKPK}$) refers to a protocol involving a prover and a verifier. In this protocol, the prover convinces the verifier that they possess a specific quantity w satisfying a computable relation R without revealing any information about w.

In our work, we adopt Camenisch and Stadler's [12] approach to representing proofs of knowledge for discrete logarithms, as well as proofs of the validity of statements concerning discrete logarithms. For instance, the expression $\mathcal{ZKPK}(\alpha, \beta, \gamma) : y = g^\alpha h^\beta \wedge \tilde{y} = \tilde{g}^\alpha \tilde{h}^\gamma$ denotes a Zero-Knowledge Proof of Knowledge for integers α, β, and γ such that the equations $y = g^\alpha h^\beta$ and $\tilde{y} = \tilde{g}^\alpha \tilde{h}^\gamma$ hold. Here, $y, g, h, \tilde{y}, \tilde{g}$, and $\tilde{h}$ are elements of certain groups $G = \langle g \rangle = \langle h \rangle$ and $\tilde{G} = \langle \tilde{g} \rangle = \langle \tilde{h} \rangle$. Conventionally, Greek letters represent the quantities for which knowledge is being proved, while other parameters are known to the verifier.

B Security Proofs

Proof of Theorem 1. Subsequence membership. Let, the advantage of the adversary in winning the game is non-negligible. Based on the winning conditions, there must exist an entry $(\pi, S, \mathbf{y}) \in L_{verify}$, that is, the list of credentials $\mathbf{y}$ output by the adversary was successfully verified to be a subsequence of a set of credentials represented by snapshot s. Let $\mathbf{x} = \{x_i, ..., x_n\}$ be the set of credentials represented by s.

Let (y_i, y_j) be the pair of credentials in $\mathbf{y}$, that also exists in the set $\mathbf{x}$, but in a different order, as required by the winning condition. This can only happen if at least one of the following cases is satisfied:

- Case 1: The list L_{issue} maintained by the challenger C stores credentials in wrong issuing order. In our proposed construction, this implies the snapshot manager appending credentials in the buffer Q in the wrong order. However, this could only happen if either (1a) the credential issuer has sent credentials (y_i, y_j) in the wrong order, or (1b) the snapshot manager has received these credentials in the wrong order due to uneven delay on the communication channel, or (1c) at least one of the credentials in (y_i, y_j) that SM believed to be sent by the credential issuer, was actually sent by the adversary.
  Case (1a) contradicts the assumption the credential issuer sends credentials immediately after generation.
  Case (1b) contradicts the assumption that all the credentials are received by the snapshot manager instantaneously.
  Case (1c) can happen if either one of the credentials in (y_i, y_j) was forged by the adversary, contradicting our assumption that the digital signature

scheme $\mathbb{DS}$ is resistant against existential forgery. Or, the adversary could simply replay one of the credentials to the snapshot manager at a later time. However, this is prevented by the credential issuer embedding session IDs in each credential, thus guaranteeing freshness.

– Case 2: $\texttt{Verify}(\pi, S, \mathbf{y})$ accepts a false Merkle inclusion proof. However, this attack can be reduced to breaking the collusion resistance property of the hash function H, or, breaking the computational binding property of the $\mathbb{PC}$ scheme ($\mathbb{P}\circledcirc\mathbb{C}\$+$) contradicting our assumption.

Selective reveal. Let, the advantage of the adversary in winning the game is non-negligible. That implies A was able to determine the i-the message by looking at the rest of $(N-1)$ messages. Since all the message bits are independent of each other, the only way A could guess the bit with non-negligible probability is by breaking the pre-image resistance of the hash to reveal the content of the leaf node or Merkle Hash tree or breaking the computational hiding property of $\mathbb{PC}$ scheme ($\mathbb{P}\circledcirc\mathbb{C}\$+$). Both are contradictions.

Proof of Theorem 2. Integrity: Let us assume that the advantage of the adversary is non-negligible. We can have two cases.

Case 1: Winning condition (i) is fulfilled, that is, There exists a $pol \in \mathbf{y}$ such that it was not generated or issued by a trusted issuer, i.e., $(pol, ., .) \notin L_{issue}$. This can be done in 2 ways:

a) the token *pol* was not generated at all by a trusted issuer. This, however, is the winning condition for the unforgeability game for the $\mathbb{P}\circledcirc\mathbb{L}$ scheme. Following theorem 3, we can conclude that the probability of A satisfying the winning condition (i) is negligible.

(b) the token *pol* was generated by a trusted issuer but for a different user U'. This, however, is the winning condition for the non-transferability game for the $\mathbb{P}\circledcirc\mathbb{L}$ scheme. Following theorem 3, we can conclude that the probability of A satisfying the winning condition (i) is negligible.

Case 2: Winning condition (ii) is fulfilled, that is, There exists a $pol \in \mathbf{y}$ such that it was issued to a different user, i.e., $(pol, U', .) \in L_{issue}$, $U' \neq U$.

Therefore, the provenance authority did not update the snapshot for user U on proof-of-location *pol*. Since the provenance authority is trusted, there can be two different ways that this condition is satisfied. a) *Adv* manipulated the accumulator A on the distributed ledger. But as we assumed the distributed ledger to be resistant against tampering, and that only authorized parties can write into it, this is a contradiction. b) *Adv* convinced the verifier in step (v) of `VerProv` the snapshot $H(s')$ is a member of accumulator A, such that setp (vi) is successfully verified as well for this particular snapshot s'. This is possible only either by breaking the strong collision resistance property of the accumulator A, or by breaking the collision resistance property of hash function H, and both are contradictions.

Case 3: Winning condition (iii) is fulfilled, that is, There exists a $pol \in \mathbf{y}$ such that it was not the input of any `Update` protocol run, i.e., $(., ., pol, .) \notin L_{update}$.

This is possible if either a) snapshot for including pol_j was updated before the snapshot for pol_i was updated. This, however, is the winning condition for the adversary in the $\mathbb{P}\circ\mathbb{C}\$$ Game for subsequence membership, and following theorem 1, the winning probability for the adversary is negligible.

b) indexes (i) associated with each *pol* were manipulated after the snapshots were accumulated in A. This is only possible by breaking the strong collision resistance property of the accumulator A.

c) Setp (vi) of `Verify` is successfully verified for a manipulated snapshot s', that represents manipulated indexes for pol_i, pol_j. This is again, the winning condition for the adversary in the $\mathbb{P}\circ\mathbb{C}\$$ Game, and following theorem 1, the winning probability for the adversary is negligible.

Unlinkability: Adversary A is given with output of $Verify$ oracle as well as transcripts of both verification sessions (indexed with $j \in 1, 2$), that include $s^j, \pi_4^j, \pi_5^j, \pi_6^j, \{\langle k_i^j, pol_i^j\rangle : i = 1 \ldots n\}$.

Let, the adversary won the game, that is, they could successfully determine whether the two sets of *pol*s S_1, S_2, from two sessions, belong to the same user or different users. This is possible, if:

a) Adversary can determine if π_4^1 (from session 1) and π_4^2 (from session 2) belong to the same user or not. π_4 is a zero-knowledge proof of knowledge of the statement: user knows the secret committed in all commitments $com_1, \ldots, com_n$. To learn any information other than this statement, the adversary would have to break the zero-knowledge property of the $\mathbb{ZKPK}$ scheme, which is a contradiction.

b) Adversary can determine if π_5^1 (from session 1) and π_5^2 (from session 2) belong to the same user or not. π_5 is a zero-knowledge proof of knowledge of the statement: user knows the secret witness w for s, such that s is a member of the accumulator. To learn any information other than this statement, the adversary would have to break the zero-knowledge property of the $\mathbb{ZKPK}$ scheme, which is a contradiction.

c) Adversary can determine if snapshots s^1 (from session 1) and s^2 (from session 2) belong to the same user or not.

d) Adversary can determine if $\{\langle k_i^j, pol_i^j\rangle : i = 1 \ldots n\}$ (for $j = 1, 2$) of the two sessions (indexed by j) belong to the same user or not. Since the two sessions do not overlap in case of the verifier is the same, there are no common *pol*s in the two sets S_1, S_2, and cannot be used by the adversary. In case the two sets overlap for two colluding verifiers V_1, V_2, the adversary must compare the signature part of the common *pol*, but this cannot be done because of the DV *non-transferability privacy* property.

Proof of Theorem 3. The proof follows from [4]. We follow their security model, and our construction, including the distance bounding protocol, is similar to their construction, with the only exception of the use of a designated verifier signature scheme in place of the digital signature scheme. Replacing $\mathbb{DS}$ with $\mathbb{DVS}$ however does not compromise security, since $\mathbb{DVS}$ provides the required property of $\mathbb{DS}$, which is existential unforgeability.